

Working with Arithmetic Operator :

Division & Modulus Operator :

```
OpPtr3.java
-----
void main()
{
    IO.println(10/3); //3
    IO.println(10/5); //2
    IO.println(5/5); //1
    IO.println(4/5); //0
    IO.println("=====");
    IO.println(10 % 3); //1
    IO.println(10 % 2); //0
    IO.println(4 % 5); //4
}
```

WAP to find out the reverse of a 3 digit number (/ and %)

```
OpPtr4.java
-----
void main()
{
    int num = Integer.parseInt(IO.readLine("Enter a 3 digit number :")); //567

    int rem = num % 10; //rem = 7

    IO.print("Reverse of "+num+" is :"+rem); //765

    num = num /10; //num = 56

    rem = num % 10; //rem = 6

    IO.print(rem);

    num = num / 10; //num = 5

    IO.print(num);

    IO.println();
}
```

Unary Operator:

The operator which works upon **single operand** is called Unary Operator. The following are the basic unary operator.

- a) ! : It is logical complement Operator. It will invert the boolean value. [true -> false and false -> true]
- b) + : It is unary + operator. [by default, number is positive only]
- c) - : It is unary - Operator.
- d) ++ : It is unary increment operator, Increments the value by 1
- e) -- : It is unary decrement operator, Decrements the value by 1

OpPtr5.java

```
-----
void main()
{
    boolean isEligible = true;
    IO.println(isEligible);
    IO.println(!isEligible); //Logical Complement Operator
}
```

OpPtr6.java

Unary Minus Operator

```
void main()
{
    int x = 15;
    IO.println(-x); //-15
    IO.println(-(-x)); //15
}
```

Working with Unary Increment & Decrement Operator :

| Initial Value of x | Expression               | Value of y | After evaluation the value of x |
|--------------------|--------------------------|------------|---------------------------------|
| int x = 10;        | int y = ++x; //Pre Incr  | 11         | 11                              |
| int x = 10;        | int y = x++; //Post Incr | 10         | 11                              |
| int x = 10;        | int y = --x; //Pre Decr  | 9          | 9                               |
| int x = 10;        | int y = x--; //Post Decr | 10         | 9                               |

MANTRA :

++x; [Always use the new value of x]      [The same rule is applicable for decrement operator also]  
x++; [Always use the old value of x]

OpPtr7.java

```
-----
void main()
{
    int x = 15;
    int y = x++; //First assignment then increment
    IO.println(x+" "+y);
}
```

OpPtr8.java

```
-----
void main()
{
    int x = 15;
    int y = ++15; //Error [Increment & decrement Operator we can apply on variables but not constants]
    IO.println(y);
}
```

OpPtr9.java

```
-----
void main()
{
    int x = 15;
    int y = ++(++x);
    IO.println(y);
}
```

OpPtr10.java

```
-----
void main()
{
    char ch ='A';
    ch++;
    IO.println(ch); //B
}
```

OpPtr11.java

```
-----
void main()
{
    double d = 15.15;
    d++;
    IO.println(d); //16.15
}
```

Note : Increment & decrement operator we can apply with any data type except boolean

OpPtr12.java

```
-----
void main()
{
    int amount = 0;
    IO.println(amount); //0
    IO.println(++amount); //1
    IO.println(amount); //1
    IO.println(amount--); //1
    IO.println(amount); //0
}
```

Note : Increment & decrement operator is having high order of precedence as compared to binary operator.

OpPtr13.java

```
-----
void main()
{
    int i = 5; //6
    int j = i++ + 4; // 5 + 4
    IO.println(i+" : "+j);
}
```

OpPtr14.java

```
-----
void main()
{
    int i = 5; //6
    int j = ++i + 4;
    IO.println(i+" : "+j);
}
```

OpPtr15.java

```
-----
void main()
{
    int i = 5; //4
    int j = --i - 4;
    IO.println(i+" : "+j);
}
```

OpPtr16.java

```
-----
void main()
{
    int x = 0; //0
    x += x++; //x = x + x++; // 0 + 0
    IO.println(x);
}
```

OpPtr17.java

```
-----
void main()
{
    int i = 5; //7
    int result = i++ + ++i; // 5 + 7

    IO.println("i = " + i);
    IO.println("result = " + result);
}
```

OpPtr18.java

```
-----
void main()
{
    int a = 5; //7
    int b = a++ + a++; //5 + 6

    IO.println(a);
    IO.println(b);
}
```

OpPtr19.java

```
-----
void main()
{
    int x = 10;
    int y = ++x + x++;

    IO.println(x);
    IO.println(y);
}
```

OpPtr20.java

```
-----
void main()
{
    int x = 15; //17
    IO.println(++x + x++); //32
    IO.println(x); //17
    IO.println(".....");

    int y = 15; //17
    IO.println(++y + ++y); //33
    IO.println(y); //17
}
```

OpPtr21.java

```
-----
void main()
{
    int x = 15;

    x += x++; //x = x + x++; // x = 15 + 15
    IO.println(x);
}
```

OpPtr22.java

```
-----
void main()
{
    int x = 3; //3
    int y = ++x * 5 / x--; //4 * 5 / 4
    IO.println(x+" : "+y); //3 : 5
}
```

OpPtr23.java

```
-----
void main()
{
    int a = 4; //4
    int b = a++ * 2 / --a; //4 * 2 / 4
    IO.println(a+" : "+b);
}
```

OpPtr24.java

```
-----
void main()
{
    int a = 6; //7
    int b = a++ + a-- + ++a; //6 + 7 + 7
    IO.println(a+" : "+b);
}
```

=====

Relational Operator :-

These operators are used to compare the values. The return type is boolean.

- 1) > (Greater than)
- 2) < (Less than)
- 3) >= (Greater than or equal to)
- 4) <= (Less than or equal to)
- 5) == (double equal to)
- 6) != (Not equal to )
- 7) instanceof operator [We will learn in Polymorphism concept]

OpPtr25.java

```
-----
void main()
{
    int a = 10;
    int b = 20;
    IO.println("a == b : " + (a == b) );
    IO.println("a != b : " + (a != b) );
    IO.println("a > b : " + (a > b) );
    IO.println("a < b : " + (a < b) );
    IO.println("b >= a : " + (b >= a) );
    IO.println("b <= a : " + (b <= a) );
}
```

=====

CONTROL STATEMENTS IN JAVA:

In Java, Basically A Java program executes from top to bottom but by using control statements we can control the flow of execution of a java program.

It helps to decide which code should run, when it should run, and how many times it should run.

Block.java

```
-----
void main()
{
    {
        IO.println("Block 1");
    }

    {
        IO.println("Block 2");
    }

    {
        IO.println("Block 3");
    }
}
```

Control statements fall into three main categories:

- 1.Selection OR Decision-Making Statements
  - a) if statement
  - b) if-else statement
  - c) else-if statement
  - d) Nested if
  - e) switch statement (JDK 17V with yield as a return value)
- 2.Iteration OR Looping Statements
  - a) do-while loop
  - b) while loop
  - c) for loop
  - d) for each loop (Enhanced for loop)
- 3.Jumping Or Branching statement
  - a) break
  - b) continue
  - c) return
  - d) yield

If condition:

It is decision making statement.  
It is used to test a boolean expression.  
The expression must return boolean type.  
It allows us to execute a block of code only when a given condition is true.  
If the condition returns to false, the control skips the if block and moves to the next statement.  
The if statement is used when we want to execute a block of code only if a condition is true, and does not perform any action, if condition is false.

Syntax:

```
if(condition)
{
    // statements executed when condition is true
}
```

Programs: [Real time examples]

a) Displaying a low battery warning

if1.java

```
-----
void main()
{
    int batteryPercentage = Integer.parseInt(IO.readLine("Enter your battery percentage :"));

    if(batteryPercentage <=20)
    {
        IO.println("Battery is low!!!!");
    }
}
```